

The Vodka Render Report

Why every click rebuilds the document, why the last fix didn't work, and the plan that will.

2026-09-04 · four investigation agents + measurements · branch pr-envelope stack

The short version

The slow click is three separate problems stacked, and the previous fix addressed only one of them. Selection dirties the clicked node's parent (for the insertion pip), a dirty node rebuilds its entire DOM subtree, and even a "kept" subtree that gets detached and re-appended pays the browser's full style-recalc + layout bill. `c0e5a84` (the shallow-dirty commit, tested with its flag *on*) fixed the middle problem and tested as "no effect" because the third one — layout — dominates INP. The winning move for clicks is to stop touching the parent at all: selection becomes two class toggles plus moving one pip element. Measured: **0.1ms** where today costs ~550ms.

1 · The measurements

Headless Chrome, a vodka-shaped tree (nested inline-block divs, sizes purely from CSS), 137k nodes, clicked node's parent owning a 19.6k-node subtree:

APPROACH	TIME
Today: reuse divs, wipe contents, recurse (what <code>render()</code> does)	549 ms
Keep flag on children, but still detach/re-append them ($\approx$ <code>c0e5a84</code>)	75 ms
No detach: rewrite class/style on the affected nodes only	0.1 ms
Pure JS walk of 100k nexes comparing cached fields	1.8 ms
Reading one layout property (<code>offsetTop</code>) on 100k nodes	332 ms

Two conclusions hide in this table. Detaching and re-appending a big subtree costs a large fraction of a full rebuild, because the browser re-runs style and layout on everything reattached — that is why `c0e5a84` felt inert and why DevTools showed *Rendering* 422ms vs *Scripting* 220ms. And reading DOM geometry is as expensive as rebuilding, which permanently rules out any "measure distance from top" culling scheme inside the render path.

Layout after a detach/re-append was verified byte-identical to an untouched sibling, `:first-child` and `:nth-child` rules included: **no parent in vodka has ever needed its children's DOM to size itself**. That conception is inherited from pre-CSS vodka. The only geometry reads in the whole codebase are two builtins in `syscalls.js` and arrow-key navigation in `manipulator.js`.

2 · How it got here (git archaeology)

The "dirty = rebuild whole subtree" line is **not** accumulated scar tissue from copy/paste bugs. It was written in one commit and never touched again:

- **2020-02** — render flags born; `RENDER_FLAG_SHALLOW` means "redraw this node's chrome, don't touch children" and is used for exactly the case that is slow today: selection change.
- **2020-12-19**, `dd71bbd` "implemented dirty-only rendering" — introduced `RENDER_IF_DIRTY`, wrote the flag-strip at `rendernode.js:427` in its final form, and *deleted every setter of SHALLOW* in the same 40-file commit. Selection went from "shallow-render two nodes" to "dirty the parent, rebuild its subtree." `git log -L` proves the block is byte-identical since.
- No commit ever widened dirty-marking after a bug. `manipulator.js` has zero dirty calls and never had any — structural edits dirty containers via `NexContainer.changed()`. The remembered copy/paste rendering bug is `b1194de` (2019), fixed in the copy path, pre-`RenderNode`.
- **2026-09-02** — the two perf attempts (`12655fa` viewport culling, `c0e5a84` shallow dirty) live only on `pr-repeat-duration`; neither is an ancestor of the current stack. `c0e5a84` 's flag defaulted *on*, so it was active during the "no effect" test — the honest reading is that it works as designed and the layout cost it can't avoid swamps the saving.

`RENDER_FLAG_SHALLOW` and `RENDER_FLAG_RERENDER` have been write-only for five years: ten readers across the nex types all permanently take the "not shallow" branch. One live side

effect: because `RERENDER` is never set, `nex.js:564` re-installs the mousedown handler on every node on every pass.

3 • What React actually does (corrected model)

React does not scan the tree for dirt. `setState` marks that fiber and walks *upward*, OR-ing a lane bit into every ancestor's `childLanes` — push-based marking, same spirit as vodka's. The render walk then starts at the root but **skips whole clean subtrees without visiting them**, because a clean node with empty `childLanes` proves nothing below it needs work. Commit applies a minimal recorded diff; it never rewrites an element wholesale, and it snapshots/restores focus and selection around real replacements.

Keys and the type-change-remounts heuristic exist only because React elements have no queryable identity. Vodka's nexes have stable IDs, so both are unnecessary — a per-parent `Map<nexId, RenderNode>` gives exact (not heuristic) child matching.

The closer relatives are the non-vdom frameworks: **SolidJS** (each DOM binding subscribes to its data; update cost is proportional to what changed, no diffing) and **Lit** (cache what you last committed, write only deltas). Vodka's shape — the data tree *is* the source tree, every node owns its div, mutators already push dirty bits — is Solid's model at `RenderNode` granularity. The discipline to import: a change patches exactly the DOM it owns; "tear down" applies only to nodes that actually left the tree.

4 • The plan

1 Surgical fixes — small, independent, each worth an order of magnitude

- **Selection stops dirtying the parent.** There is only ever one selection and one pip. `setSelected / setUnselected / setInsertionMode` toggle `newselected` on the two nodes and remove/insert the single pip element relative to `selectedNode.getDomNode()`, instead of dirtying parents. `INSERT_AROUND`'s wrapper square is the one hard case (it reparents one node) — keep the dirty-parent path as its fallback initially. Clicks become ~4 DOM ops, independent of document size. *This is the INP fix.*
- **Wavetable interactions stop rebuilding the document.** 18 call sites in `wavetable.js` + `wtmanip.js` call `enqueueTopLevelRender()` — an unconditional

full-document render — per scrub, marker, or record tick. Substitute the existing `renderOnlyThisNex()`. Trivial and huge at scale.

- **Kill the $O(c^2)$ child walk.** `NexContainer` children are a linked list; `getChildAt(i)` walks from the head, and `render()` calls it once per loop iteration (`rendernode.js:486`, again in `renderNewChildAt`, again in `line.js` 's emptyline check). A 5,000-child container pays ~12.5M pointer hops per rebuild. Walk the existing iterator alongside the index. Makes the deep rebuilds that legitimately remain (command evaluation) cheap too.

2 Re-land shallow-dirty ("Part A") with the audit's three guards

Cherry-pick `c0e5a84` 's idea — a dirty node keeps `RENDER_IF_DIRTY` on its children so clean subtrees skip — which still matters for structural edits mid-document once Phase 1 removes selection from the picture. The correctness audit verified its structural invariants hold (every child-list mutation dirties the container via `changed()`; mode flips `rendernode-dirty` whole subtrees via `setRenderMode` 's recursion) and found exactly three must-have guards:

- **GUARD** `setInsertionMode` must dirty **self and parent** (today: parent only) — the node's own render draws the `INSERT_INSIDE` pip and the `leftinsert / rightinsert` ... classes in letter/line/word.
- **GUARD** Editor **start** must dirty the edited node itself (`startEditor` dirties only the parent; only estring self-dirties). Otherwise no tilde/editing chrome until the first keystroke.
- **GUARD** `setOnNextRenderCallback` must `setDirtyForRendering(true)`, or the `on-next-render` deferred hangs forever on a clean nex.

Known residue, pre-existing and merely widened: object-tag staleness for a nex aliased in two places when only one occurrence re-renders.

3 Remove the $O(\text{document})$ floor per keystroke

- **Upward "descendant dirty" bit** (React's `childLanes`): marking dirty also flags ancestors, so the render walk prunes clean subtrees instead of visiting all ~137k `RenderNodes` to ask each one.
- **Replace `setAllNotDirty()`** (`vodka.js:182` walks the whole tree after every pass) with clearing bits as nodes render, or an explicit dirty set.
- **Org's dirty check is expensive and broken:** `org.js` `getDirtyForRendering` walks its children twice per visit building two `Tag` objects — and `new`

`Tag(':shouldDraw')` silently ignores its argument (the `Tag` constructor takes none), so it matches only empty-string tags. The `::draw / :shouldDraw` dirty machinery has never worked as written.

- Optionally align flushes to `requestAnimationFrame`; the queue's dedupe already collapses to one render per click, so this is minor.

5 • Rejected: non-destructive re-render of a dirty node ("Part B")

The idea of reusing the vestigial `SHALLOW` path so a dirty node rewrites its own chrome without detaching children is **BREAKS** in its proposed form — not with guards, but structurally. The audit's inventory:

- `getTagHolder` *prepends a fresh tagholder every render*; object tags likewise → duplicates accumulate. `Command/lambda`'s override returns `domNode.firstChild` (position-sensitive).
- `Command/lambda/instantiator/deferredcommand` put all their content *updates* inside `!SHALLOW` blocks → preserved codespans show stale text; typing in a command editor would not display. Plus a precedence bug at `command.js:553` (`!(f&S) && a || b`).
- `deferredvalue.js:274` crashes outright under `SHALLOW` (`dotspan` used unguarded).
- `Wavetable`, `letter`, `eerror`, `clip`, `contract`, `surface` append every render relying on the destructive `clear`; `org.js:168` sets `innerHTML` on a container.
- `Pips` and `infix spans` are siblings interleaved with child divs, with no ownership tracking that would let a non-destructive pass remove stale ones.

Making this safe means auditing every `renderInto` and adding decoration bookkeeping — a mount/update split per nex type, Lit-style. That is the right *eventual* architecture (build static structure once; update only dynamic parts), but it is a project, not a patch, and Phases 1–3 get the wins without it.

6 • Side findings (bugs found along the way)

WHERE

FINDING

tag.js:24	Tag constructor ignores arguments; every <code>new Tag('...')</code> call site gets an empty tag. Breaks org's <code>::draw / :shouldDraw</code> logic. Verified.
command.js:553	Operator precedence: <code>!(flags & SHALLOW) && html != '' ghostDiv</code> appends a codespan under SHALLOW when a ghost exists. Dormant while SHALLOW is never set.
filebuiltins.js:453	<code>normal-mode / exploded-mode : nodes.forEach(node => ...)</code> never uses <code>node .</code>
rendernode.js:438	MAX_RENDER_DEPTH early return skips clearing the rendernode dirty bit → depth-exceeded nodes re-render every pass.
globalconstants.js:28	RENDER_FLAG_SHALLOW / RERENDER dead since 2020; RENDER_FLAG_DEPTH_EXCEEDED never set, and <code>nex.js:583</code> would throw if it were. Candidates for deletion once Part B is formally shelved.
nex.js:564	Click handler removed + re-added on every node every pass (dead RERENDER guard). Bind once per RenderNode, or delegate at the root — folds into Phase 3.

7 • How to verify

- **Before touching anything:** in a big doc, click a deeply nested letter vs a top-level org. Prediction: deep is already fast, top-level is slow. If both are slow, something else dirties high nodes — stop and find it.
- **After Phase 1:** INP on a click in a large doc should drop from ~800ms to <50ms; wavetable scrubbing should stop skipping.
- **After Phase 2:** exercise the three guard paths — cycle insertion modes on a selected node (pips move), open an editor on command/integer/wavetable (chrome appears immediately), run an `on-next-render`-based builtin (deferred resolves).
- Golden/test suite as usual — Jason runs it; the render changes shouldn't alter any serialized output.

Sources: four investigation agents (git archaeology of the render system, React/Svelte/Solid/Lit research, full-pipeline optimization review, adversarial correctness audit of the proposal) plus direct headless-Chrome measurement. All file:line references verified against the working tree on 2026-09-04.